

Описание архитектуры и Sizing

Channelbox

Exported on 09/19/2026

Table of Contents

Содержание	8
0. Краткий справочник: выбор конфигурации	9
Pilot – до 1 000 диалогов/мес	9
Start – до 10 000 диалогов/мес	9
Standard – 10 000 – 100 000 диалогов/мес	10
Medium – 100 000 – 200 000 диалогов/мес	11
Large – 200 000 – 300 000 диалогов/мес	11
1. Обзор	13
Назначение	13
Требования к ПО	13
Поддерживаемые платформы виртуализации	13
2. Архитектура компонентов	14
3. Список Docker-контейнеров	15
Потребление RAM по контейнерам (типичное)	16
4. Сетевая конфигурация	18
4.1. Домены	18
4.2. DNS-записи	18
4.3. Таблица портов	18
Внешние порты (открыты в firewall)	18
Внутренние порты (только localhost / Docker network)	19
4.4. IP-адреса по конфигурациям	19
4.5. Firewall (UFW)	20
Исходящий трафик	20
5. SSL-сертификаты	22
Варианты	22
Требования	22
Let's Encrypt с Certbot	22

6. Конфигурации сайзинга	23
Базовые метрики для расчёта	23
Формула расчёта хранения	23
Конфигурация Pilot — до 1 000 диалогов/мес	23
Конфигурация Start — до 10 000 диалогов/мес	24
Конфигурация Standard — 10 000 – 100 000 диалогов/мес	25
Конфигурация Medium — 100 000 – 200 000 диалогов/мес	26
Конфигурация Large — 200 000 – 300 000 диалогов/мес	27
7. Дисковое пространство — детальный расчёт	29
7.1. PostgreSQL data (SSD обязательно!)	29
7.2. Логи (HDD допустим)	29
7.3. S3 Storage (вложения, файлы)	30
7.4. Docker images + OS	30
7.5. Сводная таблица	31
8. Архитектура масштабирования	32
Level 1: All-in-one (1 сервер)	32
Level 2: App + DB (2 сервера)	32
Level 3: App + DB + S3/Logs (3–4 сервера)	33
Level 4: Кластер (5–6 серверов)	33
9. Взаимодействие слоёв	35
9.1. Потоки данных	35
9.2. Типичные потоки	35
10. Резервное копирование	37
Стратегия	37
Размер бэкапов	37
RPO / RTO	38
Пример cron для pg_dump	38
Куда складывать	38
11. Обновление системы	40

Процедура обновления	40
Downtime	40
Рекомендации.....	40
12. Внешние зависимости и SMTP	42
SMTP (обязательно для production).....	42
Внешние API-зависимости	42
13. Чек-лист для DevOps	44
13.1. Пререквизиты.....	44
13.2. Порядок установки.....	44
13.3. Валидация после установки	44
13.4. Мониторинг (рекомендации).....	45
Приложение А: Сводная таблица конфигураций	47
Краткий справочник: выбор конфигурации	49
Pilot — до 1 000 диалогов/мес.....	49
Start — до 10 000 диалогов/мес	49
Standard — 10 000 – 100 000 диалогов/мес	50
Medium — 100 000 – 200 000 диалогов/мес.....	50
Large — 200 000 – 300 000 диалогов/мес.....	51
Обзор.....	53
Назначение	53
Требования к ПО	53
Архитектура компонентов.....	54
Список Docker-контейнеров.....	55
Потребление RAM по контейнерам (типичное)	56
Сетевая конфигурация.....	58
4.1. Домены	58
4.2. DNS-записи.....	58
4.3. Таблица портов	58
Внешние порты (открыты в firewall)	58

Внутренние порты (только localhost / Docker network)	59
4.4. IP-адреса по конфигурациям	59
4.5. Firewall (UFW).....	60
Исходящий трафик	60
SSL-сертификаты	61
Варианты.....	61
Требования.....	61
Let's Encrypt с Certbot.....	61
Конфигурации сайзинга	62
Базовые метрики для расчёта	62
Формула расчёта хранения	62
Конфигурация Pilot — до 1 000 диалогов/мес	62
Конфигурация Start — до 10 000 диалогов/мес.....	63
Конфигурация Standard — 10 000 – 100 000 диалогов/мес.....	64
Конфигурация Medium — 100 000 – 200 000 диалогов/мес	65
Конфигурация Large — 200 000 – 300 000 диалогов/мес	66
Дисковое пространство — детальный расчёт	68
7.1. PostgreSQL data (SSD обязательно!)	68
7.2. Логи (HDD допустим)	68
7.3. S3 Storage (вложения, файлы)	69
7.4. Docker images + OS	69
7.5. Сводная таблица.....	70
Архитектура масштабирования	71
Level 1: All-in-one (1 сервер)	71
Level 2: App + DB (2 сервера)	71
Level 3: App + DB + S3/Logs (3–4 сервера)	72
Level 4: Кластер (5–6 серверов).....	72
Взаимодействие слоёв.....	74
9.1. Потоки данных.....	74

9.2. Типичные потоки	74
Резервное копирование	76
Стратегия	76
Размер бэкапов	76
RPO / RTO	76
Пример cron для pg_dump	77
Куда складывать.....	77
Обновление системы	78
Процедура обновления	78
Downtime	78
Рекомендации.....	78
Внешние зависимости и SMTP	80
SMTP (обязательно для production)	80
Внешние API-зависимости	80
Чек-лист для DevOps.....	82
13.1. Пререквизиты	82
13.2. Порядок установки.....	82
13.3. Валидация после установки	82
13.4. Мониторинг (рекомендации)	83
Приложение А: Сводная таблица конфигураций	84

Версия документа: 2.0

Дата: Февраль 2026

Аудитория: DevOps / System Administrators

Назначение: Планирование серверной инфраструктуры для развёртывания Self-hosted Edition

Содержание

1. [Краткий справочник: выбор конфигурации \(see page 0\)](#)
2. [Обзор \(see page 0\)](#)
3. [Архитектура компонентов \(see page 0\)](#)
4. [Список Docker-контейнеров \(see page 0\)](#)
5. [Сетевая конфигурация \(see page 0\)](#)
6. [SSL-сертификаты \(see page 0\)](#)
7. [Конфигурации сайзинга \(see page 0\)](#)
8. [Дисковое пространство — детальный расчёт \(see page 0\)](#)
9. [Архитектура масштабирования \(see page 0\)](#)
10. [Взаимодействие слоёв \(see page 0\)](#)
11. [Резервное копирование \(see page 0\)](#)
12. [Обновление системы \(see page 0\)](#)
13. [Внешние зависимости и SMTP \(see page 0\)](#)
14. [Чек-лист для DevOps \(see page 0\)](#)

0. Краткий справочник: выбор конфигурации

Этот раздел — самодостаточные карточки для быстрого выбора. Подробности по каждой конфигурации — в разделах 6–8.

Pilot — до 1 000 диалогов/мес

Архитектура: All-in-one (Level 1)

Серверов: 1

IP: 1 публичный

Домены: app.example.com + api.example.com

Сервер	vCPU	RAM	SSD	HDD
All-in-one	2	4 GB	40 GB	—

Нагрузка:

- Пиковый RPS: ~2–5
- WebSocket соединений: 10–30
- Операторов онлайн: до 5

Хранение (1 год):

- PostgreSQL: ~0.35 GB (SSD)
- Логи: ~0.5 GB (SSD)
- Файлы/S3: ~0.5 GB (SSD)

HA: Нет

Переход на Start при >800 диалогов/мес или >80% CPU sustained.

Start — до 10 000 диалогов/мес

Архитектура: All-in-one (Level 1)

Серверов: 1

IP: 1 публичный

Домены: app.example.com + api.example.com

Сервер	vCPU	RAM	SSD	HDD
All-in-one	4	16 GB	100 GB	50 GB

Нагрузка:

- Пиковый RPS: ~15
- WebSocket соединений: 50–100
- Операторов онлайн: до 20

Хранение (1 год):

- PostgreSQL: ~3 GB (SSD)
- Логи: ~5 GB (HDD)
- Файлы/S3: ~5 GB (HDD)

HA: Нет

Переход на Standard при >8 000 диалогов/мес или >80% CPU sustained.

Standard — 10 000 – 100 000 диалогов/мес

Архитектура: App + DB (Level 2)

Серверов: 2

IP: 1 публичный + 1 приватный

Домены: app.example.com + api.example.com

Сервер	vCPU	RAM	SSD	HDD
App	8	32 GB	60 GB	50 GB
DB	8	32 GB	300 GB	100 GB
Суммарно	16	64 GB	—	—

Нагрузка:

- Пиковый RPS: ~150
- WebSocket соединений: 500–2 000
- Операторов онлайн: до 200

Хранение (1 год):

- PostgreSQL: ~35 GB (SSD)
- Логи: ~50 GB (HDD)
- Файлы/S3: ~50 GB (HDD)

HA: Опционально (DB Replica)

Переход на Medium при >80 000 диалогов/мес или >70% RAM на DB-сервере sustained.

Medium – 100 000 – 200 000 диалогов/мес

Архитектура: App + DB + S3/Logs (Level 3)
Серверов: 3–4 (+ опционально DB Replica)
IP: 1 публичный + 2–3 приватных
Домены: app.example.com + api.example.com

Сервер	vCPU	RAM	SSD	HDD
App	8	32 GB	60 GB	50 GB
DB Primary	12	48 GB	500 GB	50 GB
S3/Logs	4	8 GB	50 GB	300 GB
DB Replica ¹	12	48 GB	500 GB	50 GB
Суммарно (без реплики)	24	88 GB	—	—
Суммарно (с репликой)	36	136 GB	—	—

¹ DB Replica опциональна, но рекомендуется при наличии требований к отказоустойчивости.

Нагрузка:

- Пиковый RPS: ~300
- WebSocket соединений: 2 000–5 000
- Операторов онлайн: до 500

Хранение (1 год):

- PostgreSQL: ~69 GB (SSD)
- Логи: ~100 GB (HDD)
- Файлы/S3: ~100 GB (HDD)

HA: Рекомендуется

Переход на Large при >170 000 диалогов/мес или >75% CPU на App-сервере sustained.

Large – 200 000 – 300 000 диалогов/мес

Архитектура: Кластер с LB (Level 4)
Серверов: 5–6

IP: 2 публичных + 3–4 частных

Домены: app.example.com + api.example.com

Сервер	vCPU	RAM	SSD	HDD
App 1	8	32 GB	60 GB	50 GB
App 2	8	32 GB	60 GB	50 GB
DB Primary	16	64 GB	800 GB	50 GB
DB Replica	16	64 GB	800 GB	50 GB
S3/Logs	4	8 GB	50 GB	500 GB
Суммарно	52	200 GB	—	—

Нагрузка:

- Пиковый RPS: ~450
- WebSocket соединений: 5 000–10 000
- Операторов онлайн: до 1 000

Хранение (1 год):

- PostgreSQL: ~104 GB (SSD)
- Логи: ~150 GB (HDD)
- Файлы/S3: ~150 GB (HDD)

HA: Обязательно (DB Replication + LB)

При >250 000 диалогов/мес — обратитесь к команде разработки для индивидуального проектирования.

1. Обзор

Назначение

Документ описывает требования к серверной инфраструктуре для развёртывания Self-hosted Edition в зависимости от планируемой нагрузки (количества диалогов в месяц).

Диалог — это полный цикл общения с клиентом, проходящий все стадии жизненного цикла: создание → назначение → обработка оператором/ботом → постобработка → закрытие. В среднем один диалог содержит ~11.5 сообщений.

Требования к ПО

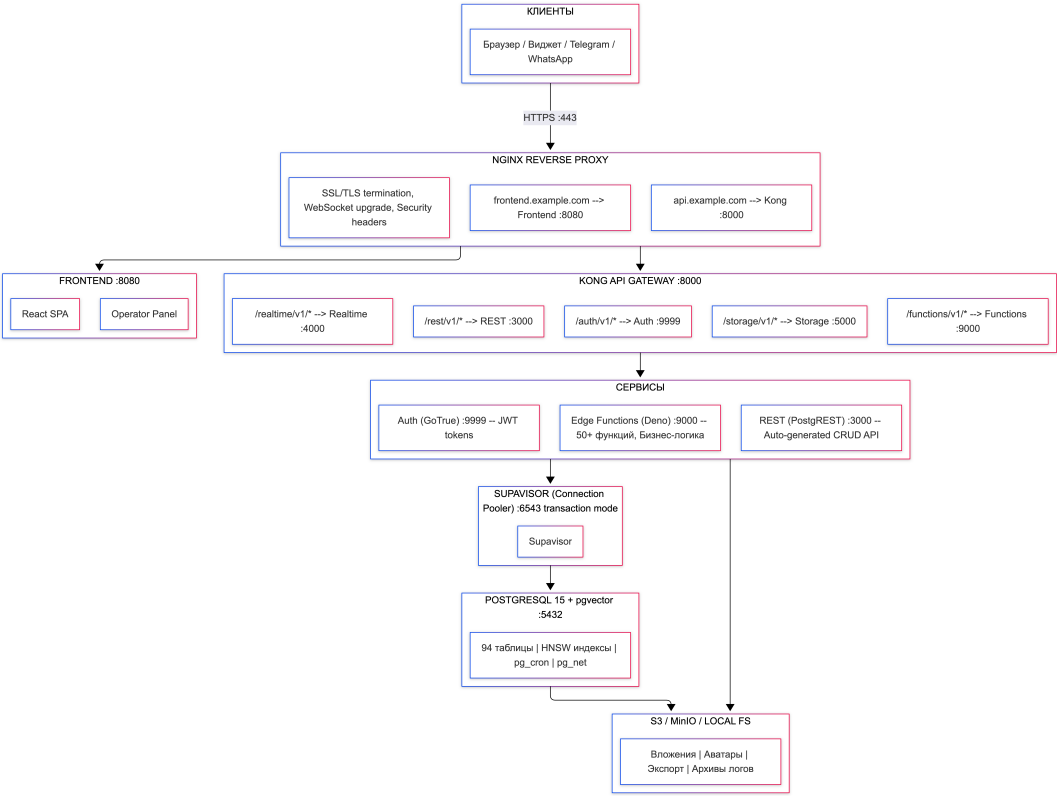
Компонент	Минимальная версия
ОС	Ubuntu 22.04 LTS / RHEL 8+ / Debian 12+
Docker Engine	24.0+
Docker Compose	v2.20+
Nginx (reverse proxy)	1.24+
Certbot (Let's Encrypt)	2.0+ (опционально)

Поддерживаемые платформы виртуализации

VMware vSphere, Nutanix AHV, Proxmox VE, KVM/QEMU, Hyper-V, Bare Metal.

GPU не требуется. LLM-инференс выполняется через внешние API-провайдеры.

2. Архитектура компонентов



3. Список Docker-контейнеров

Система состоит из **13 контейнеров**, управляемых через `docker-compose.yml`.

Конкретные версии образов указаны в `docker-compose.yml` и могут обновляться между релизами.

#	Контейнер	Образ	Порт	Доступ	Назначение
1	db	supabase/ postgres	5432	internal	PostgreSQL + pgvector, pg_cron, pg_net
2	kong	kong	8000, 8443	internal	API Gateway, маршрутизация запросов
3	auth	supabase/ gotrue	9999	internal (через Kong)	Аутентификация, JWT, OAuth
4	rest	postgrest/ postgrest	3000	internal (через Kong)	Auto-generated REST API из схемы БД
5	realtime	supabase/ realtime	4000	internal (через Kong)	WebSocket подписки на изменения БД
6	storage	supabase/ storage-api	5000	internal (через Kong)	Файловое хранилище (S3-совместимое)
7	imgproxy	darthsim/ imgproxy	8081	internal	Ресайз и оптимизация изображений
8	meta	supabase/ postgres-meta	8080	internal, без проброса ¹	Метаданные БД (для Studio)

9	functions	supabase/ edge-runtime	9000	internal (через Kong)	Deno Edge Functions (50+ функций)
10	studio	supabase/ studio	3000	internal ²	Админ-панель БД (dev/debug)
11	vector	timberio/ vector	9001	internal	Сбор и маршрутизация логов
12	supavisor	supabase/ supavisor	6543	internal	Connection pooling для PostgreSQL
13	frontend	custom build	8080	через Nginx	React SPA – операторская панель

¹ Контейнер meta использует порт 8080 внутри Docker network, наружу не пробрасывается.

Конфликта с Frontend нет, т.к. Frontend доступен через Nginx.

² Studio не выставляет порт наружу. В production рекомендуется отключать или ограничивать доступ.

Потребление RAM по контейнерам (типичное)

Контейнер	Idle	Под нагрузкой	Примечание
db	1–2 GB	4–8 GB	shared_buffers = 25% RAM
kong	150 MB	300 MB	
auth	100 MB	200 MB	
rest	80 MB	200 MB	
realtime	200 MB	500 MB	зависит от кол-ва WebSocket
storage	100 MB	300 MB	

functions	300 MB	1–2 GB	зависит от кол-ва функций
supervisor	100 MB	300 MB	
frontend	50 MB	100 MB	статика, Nginx
Остальные	200 MB	400 MB	imgproxy, meta, vector, studio
Итого	~2.5 GB	~6–12 GB	

4. Сетевая конфигурация

4.1. Домены

Для работы системы необходимо **2 домена** (или поддомена):

Домен	Назначение	Целевой сервис
app.example.com	Frontend – операторская панель	Nginx → Frontend :8080
api.example.com	API – все backend-сервисы	Nginx → Kong :8000

Рекомендуется использовать wildcard SSL-сертификат `*.example.com` для упрощения управления.

4.2. DNS-записи

```
app.example.com    A    → <IP >
api.example.com    A    → <IP >
```

При разнесённой архитектуре (Level 2+):

```
app.example.com    A    → <IP App->
api.example.com    A    → <IP App->
```

БД-сервер не требует публичного DNS, доступ по приватному IP.

4.3. Таблица портов

Внешние порты (открыты в firewall)

Порт	Протокол	Назначение
80	TCP	HTTP → redirect на HTTPS
443	TCP	HTTPS (Nginx reverse proxy)

22	TCP	SSH (администрирование)
----	-----	-------------------------

Внутренние порты (только localhost / Docker network)

Порт	Сервис	Примечание
8000	Kong HTTP	Не открывать наружу
8443	Kong HTTPS	Внутренний TLS
5432	PostgreSQL	Открыть только для DB-реплик
6543	Supervisor pooler	Transaction mode
8080	Frontend	Через Nginx
9000	Edge Functions	Через Kong
3000	PostgREST	Через Kong
9999	Auth (GoTrue)	Через Kong
4000	Realtime	Через Kong
5000	Storage API	Через Kong

4.4. IP-адреса по конфигурациям

Конфигурация	Публичных IP	Приватных IP	Примечание
Start (10K)	1	0	Всё на одном сервере
Standard (100K)	1	1	App (публичный) + DB (приватный)

Medium (200K)	1	2–3	App + DB Primary + S3/Logs (+ DB Replica)
Large (300K)	2	3–4	2× App + DB Primary + DB Replica + S3

4.5. Firewall (UFW)

```
# Базовая конфигурация
ufw default deny incoming
ufw default allow outgoing

# SSH
ufw allow 22/tcp

# HTTP/HTTPS
ufw allow 80/tcp
ufw allow 443/tcp

# PostgreSQL реплика (только для DB-сервера, только с App-)
# ufw allow from <APP_SERVER_IP> to any port 5432 proto tcp

ufw enable
```

Исходящий трафик

Контейнеры требуют исходящий HTTPS-доступ для работы с внешними сервисами:

Назначение	Адрес/порт	Контейнер
LLM API (OpenAI, Anthropic и др.)	api.openai.com:443 и аналогичные	functions
Telegram Bot API	api.telegram.org:443	functions
WhatsApp Business API	Зависит от провайдера	functions
SMTP-сервер	Настраивается, порт 587/465	auth
Docker Hub (обновления образов)	registry-1.docker.io:443	хост

Let's Encrypt (OCSP, CRL)	acme- v02.api.letsencrypt.org:443	хост
---------------------------	--------------------------------------	------

Если политика безопасности требует whitelist исходящих соединений, необходимо разрешить указанные адреса.

5. SSL-сертификаты

Варианты

Вариант	Применение	Автопродление
Let's Encrypt (wildcard)	Dev, Staging, Small Prod	Да (certbot)
Let's Encrypt (отдельные)	Если нет доступа к DNS API	Да (certbot)
Корпоративный CA	Enterprise, банки, госсектор	Ручное
Self-signed	Только для dev/testing	—

Требования

- **Wildcard** сертификат `*.example.com` покрывает оба домена
- Или **2 отдельных** сертификата: `app.example.com` + `api.example.com`
- Минимум **TLS 1.2**, рекомендуется **TLS 1.3**
- Nginx выполняет SSL termination — внутренний трафик идёт по HTTP

Let's Encrypt с Certbot

```
# Wildcard (требует DNS challenge)
certbot certonly --manual --preferred-challenges dns \
  -d "*.example.com" -d "example.com"

# Отдельные сертификаты (HTTP challenge)
certbot certonly --nginx -d app.example.com -d api.example.com
```

6. Конфигурации сайзинга

Базовые метрики для расчёта

Метрика	Значение	Источник
Средний размер диалога в БД	~16 KB	Анализ production-данных
Среднее кол-во сообщений/диалог	~11.5	Анализ production-данных
Таблицы на диалог	conversations, messages, chat_history, conversation_tags, chat_ratings, dialog_sessions, sla_events	Схема БД

Формула расчёта хранения

$DB_size = dialogues_per_month \times 16 \text{ KB} \times months \times 1.5 \text{ (индексы)} \times 1.2 \text{ (WAL/bloat)}$

Конфигурация Pilot — до 1 000 диалогов/мес

Один сервер, минимальная конфигурация для пилотных/пробных внедрений. Все компоненты на одной машине, HDD не требуется.

Параметр	Значение
vCPU	2
RAM	4 GB
SSD	40 GB (PostgreSQL data, Docker, OS, логи, файлы)

HDD	– (не требуется)
Пиковый RPS	~2–5
Concurrent WebSocket	~10–30
Операторов онлайн	до 5

Расчёт хранения за 1 год:

Тип данных	Размер/год
PostgreSQL (диалоги + индексы + WAL)	~0.35 GB
Логи (cron, API, webhook)	~0.5 GB
S3/файлы (вложения, аватары)	~0.5 GB
Docker images	~5 GB
OS + инструменты	~10 GB
Итого	~16 GB (всё на SSD)

Конфигурация Start – до 10 000 диалогов/мес

Один сервер, все компоненты на одной машине.

Параметр	Значение
vCPU	4
RAM	16 GB
SSD	100 GB (PostgreSQL data, Docker, OS)

HDD	50 GB (логи, бэкапы)
Пиковый RPS	~15
Concurrent WebSocket	~50–100
Операторов онлайн	до 20

Расчёт хранения за 1 год:

Тип данных	Размер/год
PostgreSQL (диалоги + индексы + WAL)	~3 GB
Логи (cron, API, webhook)	~5 GB
S3/файлы (вложения, аватары)	~5 GB
Docker images	~5 GB
OS + инструменты	~10 GB
Итого	~28 GB (SSD: ~18 GB, HDD: ~10 GB)

Конфигурация Standard – 10 000 – 100 000 диалогов/мес

2 сервера: App + DB.

Параметр	App-сервер	DB-сервер
vCPU	8	8
RAM	32 GB	32 GB
SSD	60 GB	300 GB ¹

HDD	50 GB	100 GB
------------	-------	--------

¹ Расчётное потребление PostgreSQL за первый год при 100K диалогов/мес — ~35 GB данных + ~17 GB индексы/WAL ≈ 52 GB. Рекомендуемый объём 300 GB включает запас на рост до 3 лет.

Параметр	Значение
Пиковый RPS	~150
Concurrent WebSocket	~500–2 000
Операторов онлайн	до 200

Расчёт хранения за 1 год (при 100K диалогов/мес):

Тип данных	Размер/год
PostgreSQL (диалоги + индексы + WAL)	~35 GB
Логи (cron, API, webhook)	~50 GB
S3/файлы (вложения, аватары)	~50 GB
Docker images	~5 GB (×2 сервера)
OS + инструменты	~10 GB (×2 сервера)
Итого	~165 GB (распределено между серверами)

Конфигурация Medium — 100 000 – 200 000 диалогов/мес

3–4 сервера: App + DB Primary + S3/Logs (+ опционально DB Replica).

Сервер	vCPU	RAM	SSD	HDD
App (Kong, Functions, Auth, Frontend)	8	32 GB	60 GB	50 GB

DB Primary (PostgreSQL)	12	48 GB	500 GB	50 GB
S3/Logs (MinIO + Vector)	4	8 GB	50 GB	300 GB
DB Replica (опционально, для HA)	12	48 GB	500 GB	50 GB

Параметр	Значение
Пиковый RPS	~300
Concurrent WebSocket	~2 000–5 000
Операторов онлайн	до 500

Расчёт хранения за 1 год (при 200K диалогов/мес):

Тип данных	Размер/год
PostgreSQL (диалоги + индексы + WAL)	~69 GB
Логи (cron, API, webhook)	~100 GB
S3/файлы (вложения, аватары)	~100 GB
Docker images	~5 GB (×3–4 сервера)
OS + инструменты	~10 GB (×3–4 сервера)
Итого	~320 GB (распределено)

Конфигурация Large – 200 000 – 300 000 диалогов/мес

5–6 серверов: 2× App + DB Primary + DB Replica + S3/Logs. Load Balancer обязателен.

Сервер	vCPU	RAM	SSD	HDD
--------	------	-----	-----	-----

App 1	8	32 GB	60 GB	50 GB
App 2	8	32 GB	60 GB	50 GB
DB Primary	16	64 GB	800 GB	50 GB
DB Replica	16	64 GB	800 GB	50 GB
S3/Logs (MinIO + Vector)	4	8 GB	50 GB	500 GB

Параметр	Значение
Пиковый RPS	~450
Concurrent WebSocket	~5 000–10 000
Операторов онлайн	до 1 000

Расчёт хранения за 1 год (при 300K диалогов/мес):

Тип данных	Размер/год
PostgreSQL (диалоги + индексы + WAL)	~104 GB
Логи (cron, API, webhook)	~150 GB
S3/файлы (вложения, аватары)	~150 GB
Docker images	~5 GB (×5–6 серверов)
OS + инструменты	~10 GB (×5–6 серверов)
Итого	~490 GB (распределено)

7. Дискковое пространство — детальный расчёт

7.1. PostgreSQL data (SSD обязательно!)

Формула:

$DB_size = (dialogues_per_month \times 16\text{ KB} \times months) \times 1.5 \text{ (индексы)} \times 1.2 \text{ (WAL/bloat)}$

Пример для 100K диалогов/мес, 12 :

$100,000 \times 16\text{ KB} \times 12 = 19.2\text{ GB}$ (чистые данные)
 $\times 1.5$ (индексы, включая HNSW pgvector) = 28.8 GB
 $\times 1.2$ (WAL, bloat, vacuum) = ~34.5 GB
+ системные таблицы, конфиги: ~2 GB
Итого: ~35 GB /

Почему SSD: PostgreSQL с расширением pgvector активно использует random I/O для поиска по HNSW/IVFFlat индексам. На HDD латентность запросов RAG увеличивается в **5–10x**.

7.2. Логи (HDD допустим)

Тип логов	Источник	Размер / 10K диалогов/мес
Cron job logs	cron_job_logs	~500 MB/год
API logs	external_api_logs	~2 GB/год
Webhook logs	event_webhook_logs	~1 GB/год
Channel logs	external_channel_logs	~1 GB/год
Vector (Docker logs)	stdout/stderr всех контейнеров	~500 MB/год
Итого		~5 GB/год на 10K

Ротация логов:

Политика	Хранение	Рекомендация
Минимальная	7 дней	Dev/Test
Стандартная	30 дней	Production
Расширенная	90 дней	Compliance, банки

7.3. S3 Storage (вложения, файлы)

Тип файлов	Средний размер	Доля диалогов с файлами
Изображения (скриншоты)	200–500 KB	~15%
Документы (PDF, DOCX)	500 KB – 2 MB	~5%
Аватары операторов	50–100 KB	фиксированно
Экспорт отчётов	1–10 MB	по запросу

Формула:

$$S3_size = dialogues_per_month \times 0.15 \times 300 \text{ KB (avg image)} \\ + dialogues_per_month \times 0.05 \times 1 \text{ MB (avg doc)}$$

Пример для 10K/:

$$10,000 \times 0.15 \times 300 \text{ KB} = 450 \text{ MB/мес ()}$$

$$10,000 \times 0.05 \times 1 \text{ MB} = 500 \text{ MB/мес ()}$$

$$\text{Итого: } \sim 1 \text{ GB/мес} \approx \sim 5 \text{ GB} / 6 \text{ мес (с учётом компрессии)}$$

7.4. Docker images + OS

Компонент	Размер
-----------	--------

Docker images (все 13 контейнеров)	~5 GB
ОС (Ubuntu 22.04 minimal)	~5 GB
Системные утилиты, certbot, monitoring	~2–3 GB
Docker overlay2 (runtime)	~3–5 GB
Итого	~15–18 GB

7.5. Сводная таблица

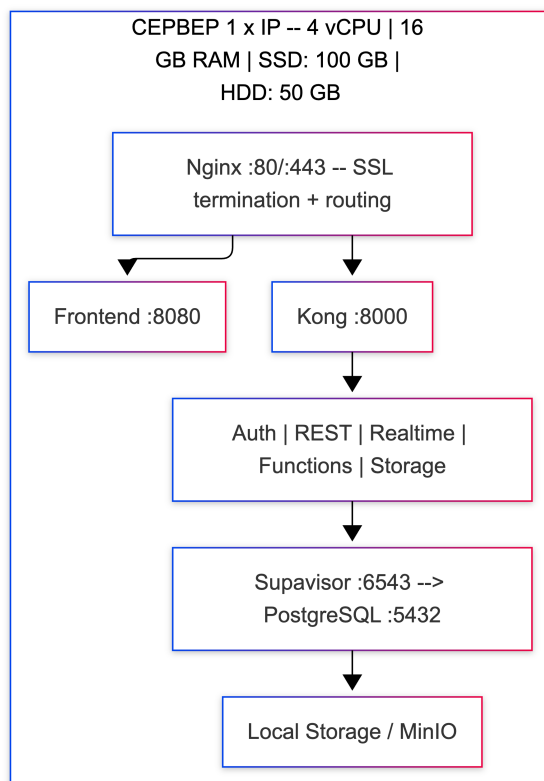
Компонент	Тип диска	Start (10K)	Standard (100K)	Medium (200K)	Large (300K)
PostgreSQL data	SSD	20 GB	100 GB	200 GB	350 GB
PostgreSQL WAL/ temp	SSD	10 GB	40 GB	80 GB	120 GB
Docker images + OS	SSD	20 GB	20 GB ×N	20 GB ×N	20 GB ×N
Логи приложения	HDD	10 GB	50 GB	100 GB	150 GB
S3 / Вложения	HDD/S3	10 GB	50 GB	100 GB	150 GB
Бэкапы БД	HDD	10 GB	50 GB	100 GB	150 GB
Итого SSD (1 сервер)		50 GB	160 GB	300 GB	490 GB
Итого HDD		30 GB	150 GB	300 GB	450 GB

N = количество серверов в конфигурации. Объёмы SSD для Standard, Medium, Large относятся к DB-серверу, где сосредоточены основные данные.

8. Архитектура масштабирования

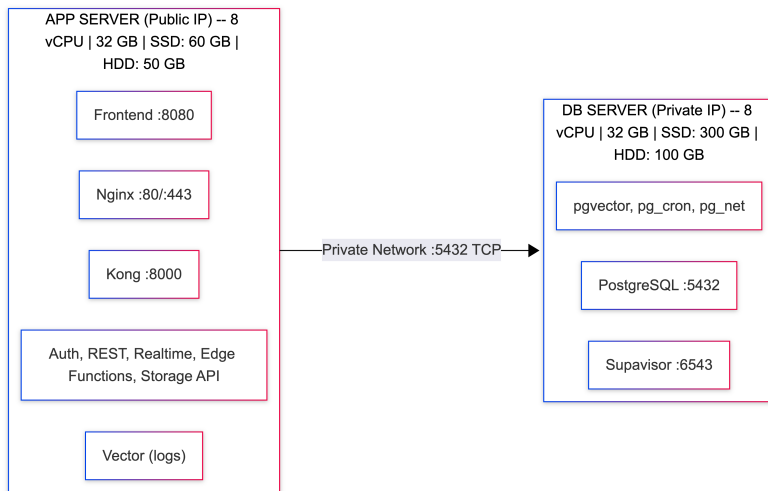
Level 1: All-in-one (1 сервер)

Конфигурация Start — до 10K диалогов/мес



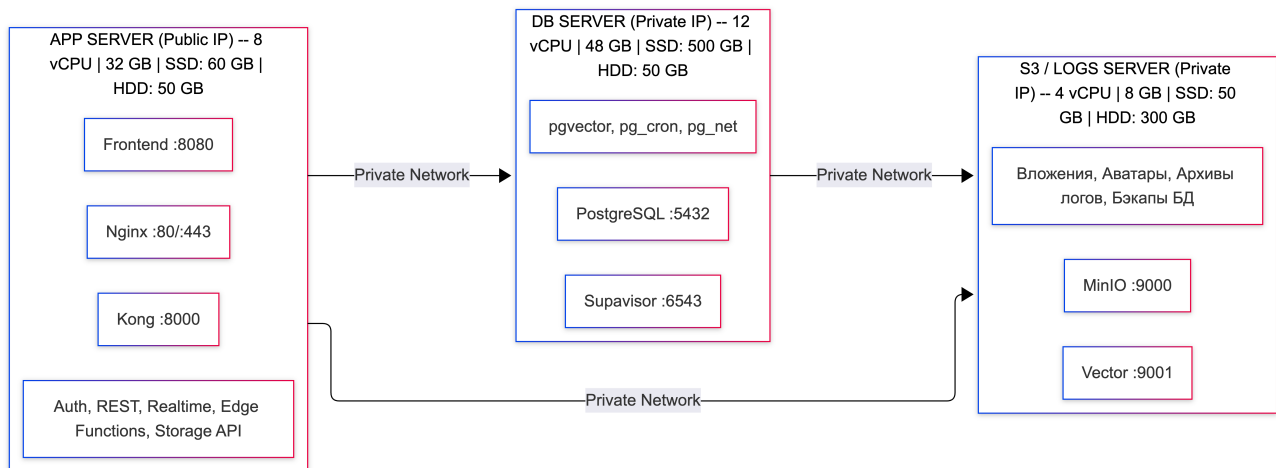
Level 2: App + DB (2 сервера)

Конфигурация Standard — 10K – 100K диалогов/мес



Level 3: App + DB + S3/Logs (3–4 сервера)

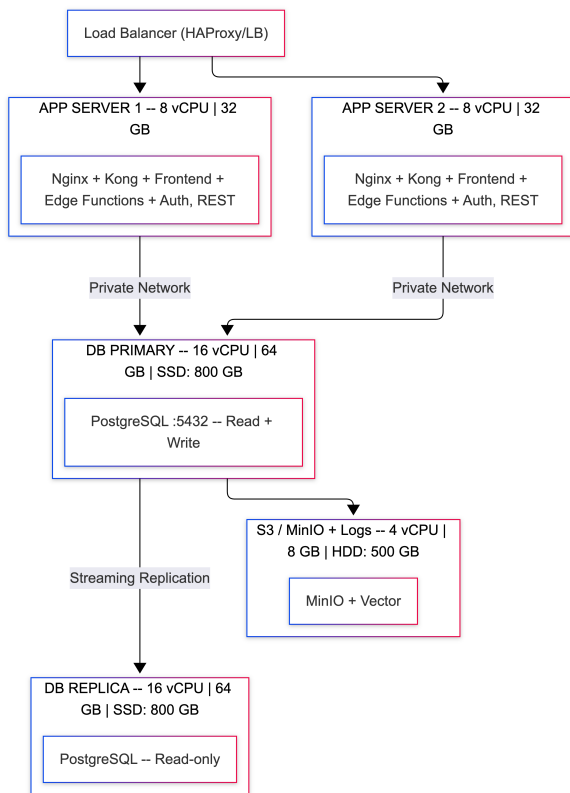
Конфигурация Medium – 100K – 200K диалогов/мес



При необходимости HA добавляется DB Replica (12 vCPU, 48 GB RAM, SSD 500 GB).

Level 4: Кластер (5–6 серверов)

Конфигурация Large – 200K – 300K диалогов/мес, HA обязателен

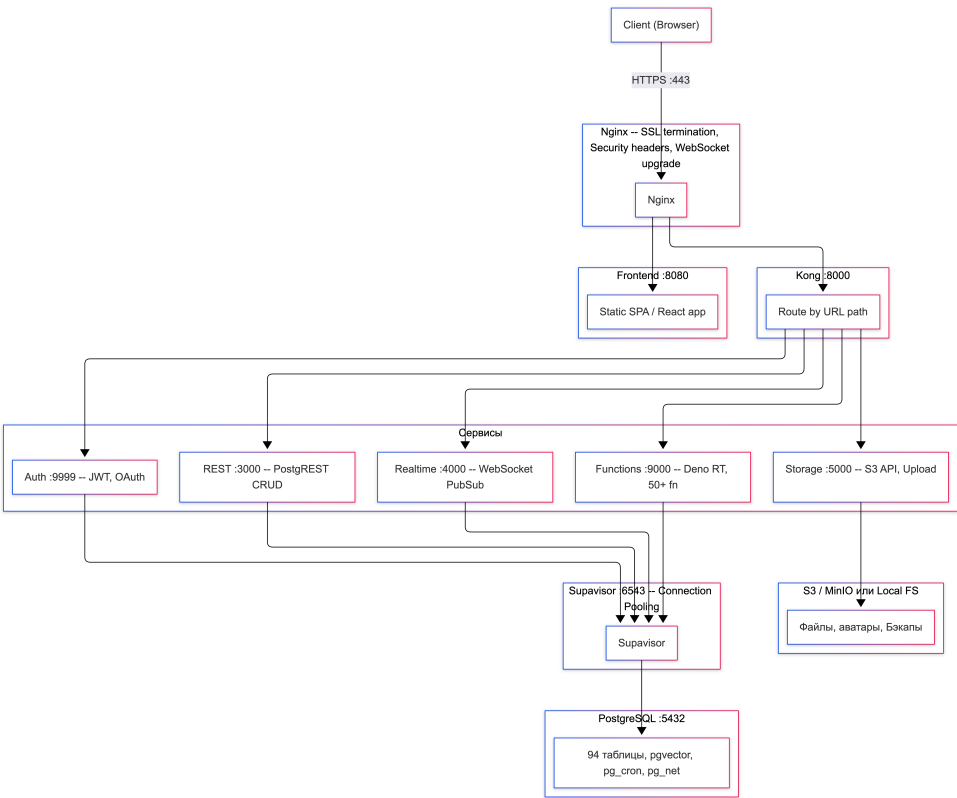


Компоненты HA:

- **Load Balancer:** HAProxy / Nginx Plus / Cloud LB
- **DB Failover:** pg_auto_failover или Patroni
- **Session Affinity:** Sticky sessions для WebSocket (Realtime)
- **Shared Storage:** MinIO на отдельном сервере

9. Взаимодействие слоёв

9.1. Поток данных



9.2. Типичные потоки

Сценарий	Путь
Оператор открывает панель	Client → Nginx → Frontend (SPA)
Загрузка списка диалогов	Client → Nginx → Kong → REST → Supervisor → PostgreSQL
Отправка сообщения	Client → Nginx → Kong → Functions → Supervisor → PostgreSQL → Realtime → Client

Загрузка файла	Client → Nginx → Kong → Storage → S3/MinIO
Входящий webhook (Telegram)	Telegram → Nginx → Kong → Functions → Supervisor → PostgreSQL
AI-ответ бота	Functions → External LLM API → Functions → PostgreSQL
Срон-задача	pg_cron → pg_net → Kong → Functions (HTTP)

10. Резервное копирование

Стратегия

Метод	Назначение	Частота	Примечание
<code>pg_dump</code>	Логический бэкап БД	Ежедневно	Подходит для Start/Standard, полная переносимость
<code>pg_basebackup</code>	Физический бэкап	Еженедельно	Для Medium/Large, быстрое восстановление
WAL archiving	Непрерывная архивация WAL	Постоянно	Point-in-time recovery (PITR)
S3 sync	Файлы и вложения	Ежедневно	<code>rclone sync</code> или <code>mc mirror</code>

Размер бэкапов

Конфигурация	Размер БД (1 год)	Размер <code>pg_dump</code> (сжатый)	Хранение 7 копий
Start (10K)	~3 GB	~1 GB	~7 GB
Standard (100K)	~35 GB	~12 GB	~84 GB
Medium (200K)	~69 GB	~23 GB	~161 GB
Large (300K)	~104 GB	~35 GB	~245 GB

RPO / RTO

Конфигурация	RPO (допустимая потеря)	RTO (время восстановления)	Метод
Start	24 часа	1–2 часа	pg_dump ежедневно
Standard	1 час	30 мин – 1 час	pg_dump + WAL archiving
Medium	~0 (минуты)	5–15 мин	Streaming Replication + PITR
Large	~0 (минуты)	5–15 мин	Streaming Replication + PITR + автофейловер

Пример cron для pg_dump

```
# /etc/cron.d/pg-backup
0 3 * * * root docker exec db pg_dump -U postgres -Fc postgres \
  | gzip > /backups/db_$(date +%Y%m%d).dump.gz

# Ротация: хранить последние 7
0 4 * * * root find /backups -name "db_*.dump.gz" -mtime +7 -delete
```

Куда складывать

Конфигурация	Основное хранилище	Рекомендация
Start	Локальный HDD раздел (/ backups)	Дополнительно: rsync на внешний сервер
Standard	Отдельный HDD раздел на DB-сервере	Дополнительно: S3/MinIO на App-сервере
Medium	S3/MinIO сервер	Рекомендуется: geo-redundant storage

Large	S3/MinIO сервер	Обязательно: geo-redundant + offsite
-------	-----------------	---

11. Обновление системы

Процедура обновления

1. **Создать бэкап БД** перед обновлением (обязательно)
2. **Остановить сервисы:**
`docker compose down`
3. **Обновить образы:**
`docker compose pull`
4. **Применить миграции БД** (если есть):
`ansible-playbook -i inventory-remote.yml playbook-remote.yml --tags database`
5. **Запустить сервисы:**
`docker compose up -d`
6. **Проверить здоровье** всех контейнеров:
`docker ps --format "table {{.Names}}\t{{.Status}}"`

Downtime

Тип обновления	Downtime	Примечание
Минорное (патч образов)	1–3 мин	<code>docker compose up -d</code> пересоздаёт изменённые контейнеры
Мажорное (миграции БД)	5–15 мин	Требуется остановки, применения миграций, перезапуска
PostgreSQL major version	30–60 мин	Требуется <code>pg_upgrade</code> или <code>dump/restore</code>

Рекомендации

- Обновление проводить в период минимальной нагрузки (ночь, выходные)
- Для конфигураций Medium/Large: обновлять App-серверы по одному (rolling update)

- Хранить предыдущие версии образов (`docker image tag`) для быстрого отката

12. Внешние зависимости и SMTP

SMTP (обязательно для production)

Для отправки email-уведомлений и подтверждений требуется SMTP-сервер.

Параметр	Переменная окружения	Пример
Хост	SMTP_HOST	smtp.example.com
Порт	SMTP_PORT	587 (TLS) или 465 (SSL)
Пользователь	SMTP_USER	noreply@example.com
Пароль	SMTP_PASS	(через vault или prompt)
Имя отправителя	SMTP_SENDER_NAME	My App
Email администратора	SMTP_ADMIN_EMAIL	admin@example.com

Без настройки SMTP не будут работать: подтверждение email, восстановление пароля, уведомления операторам.

Внешние API-зависимости

Сервис	Контейнер	Обязательность	Примечание
LLM API (OpenAI, Anthropic и др.)	functions	Опционально	Для AI-ботов и RAG
Telegram Bot API	functions	Опционально	Для Telegram-канала

WhatsApp Business API	<code>functions</code>	Опционально	Для WhatsApp-канала
SMTP-сервер	<code>auth</code>	Обязательно	Email-уведомления
Docker Registry	хост	При обновлениях	Загрузка новых образов

При наличии жёстких политик безопасности (whitelist исходящих соединений) необходимо разрешить доступ к указанным API.

13. Чек-лист для DevOps

13.1. Пререквизиты

- ☐ Сервер(а) подготовлены согласно конфигурации (CPU, RAM, диски)
- ☐ ОС установлена: Ubuntu 22.04 LTS / RHEL 8+
- ☐ Docker Engine 24+ установлен
- ☐ Docker Compose v2+ установлен
- ☐ DNS-записи настроены для обоих доменов
- ☐ SSL-сертификаты получены (или настроен Certbot)
- ☐ SMTP-сервер настроен и доступен
- ☐ Firewall настроен (UFW: 22, 80, 443 + исходящий трафик)
- ☐ SSH-доступ по ключу (пароль отключён)
- ☐ NTP синхронизация включена
- ☐ Swap отключён или минимизирован (для PostgreSQL)

13.2. Порядок установки

1. **Настройка ОС:** обновление пакетов, установка Docker, firewall
2. **SSL-сертификаты:** получение и размещение
3. **Конфигурация:** заполнение переменных окружения (`group_vars/all.yml`)
4. **Запуск Ansible:**
`ansible-playbook -i inventory-remote.yml playbook-remote.yml`
5. **Проверка здоровья:** healthcheck всех 13 контейнеров
6. **Первичная настройка:** создание админа, настройка очередей, каналов
7. **Настройка бэкапов:** cron для pg_dump, ротация
8. **Мониторинг:** настройка сбора метрик (опционально)

13.3. Валидация после установки

Проверка	Команда / URL
----------	---------------

Docker контейнеры	<code>docker ps --format "table {{.Names}}\t{{.Status}}"</code>
Kong API	<code>curl -s https://api.example.com/rest/v1/ -H "apikey: <ANON_KEY>"</code>
Frontend	<code>https://app.example.com</code> – должна загрузиться SPA
Auth	<code>curl -s https://api.example.com/auth/v1/health</code>
Realtime	WebSocket подключение через браузер
PostgreSQL	<code>docker exec -it db psql -U postgres -c "SELECT version();"</code>
SSL	<code>openssl s_client -connect api.example.com:443 -brief</code>
Edge Functions	<code>curl -s https://api.example.com/functions/v1/health-check</code>
SMTP	Отправка тестового email через Auth

13.4. Мониторинг (рекомендации)

Метрика	Инструмент	Порог
CPU usage	node_exporter + Prometheus	> 80% sustained
RAM usage	node_exporter	> 85%
Disk I/O	iostat / node_exporter	await > 20ms (SSD)

PostgreSQL connections	pg_stat_activity	> 80% max_connections
PostgreSQL replication lag	pg_stat_replication	> 1 MB
Docker container restarts	docker_events	any restart
SSL cert expiry	certbot / blackbox_exporter	< 14 days

Приложение А: Сводная таблица конфигураций

	Pilot (1K)	Start (10K)	Standard (100K)	Medium (200K)	Large (300K)
Диалогов/мес	до 1K	до 10K	10K – 100K	100K – 200K	200K – 300K
Серверов	1	1	2	3–4 ¹	5–6
vCPU (суммарно)	2	4	16	24–36 ¹	52
RAM (суммарно)	4 GB	16 GB	64 GB	88–136 GB ¹	200 GB
SSD (суммарно)	40 GB	100 GB	360 GB	610–1 110 GB ¹	1 770 GB
HDD (суммарно)	—	50 GB	150 GB	400–450 GB ¹	700 GB
Публичных IP	1	1	1	1	2
Приватных IP	0	0	1	2–3	3–4
Пиковый RPS	~2–5	~15	~150	~300	~450
WebSocket	10–30	50–100	500–2 000	2 000–5 000	5 000–10 000
Операторов онлайн	до 5	до 20	до 200	до 500	до 1 000
HA	Нет	Нет	Опционально	Рекомендуется	Обязательно
PostgreSQL/год	~0.35 GB	~3 GB	~35 GB	~69 GB	~104 GB

Логи/год	~0.5 GB	~5 GB	~50 GB	~100 GB	~150 GB
S3/год	~0.5 GB	~5 GB	~50 GB	~100 GB	~150 GB

¹ Диапазоны для Medium указаны без/с DB Replica. Без реплики: 3 сервера, 24 vCPU, 88 GB RAM. С репликой (рекомендуется): 4 сервера, 36 vCPU, 136 GB RAM.

По вопросам масштабирования и нестандартных конфигураций обращайтесь к команде разработки.

Краткий справочник: выбор конфигурации

Этот раздел — самодостаточные карточки для быстрого выбора. Подробности по каждой конфигурации — в разделах 6–8.

Pilot — до 1 000 диалогов/мес

Архитектура: All-in-one (Level 1)

Серверов: 1

IP: 1 публичный

Домены: app.example.com + api.example.com

All-in-one	2	4 GB	40 GB	—
------------	---	------	-------	---

Нагрузка:

- Пиковый RPS: ~2–5
- WebSocket соединений: 10–30
- Операторов онлайн: до 5

Хранение (1 год):

- PostgreSQL: ~0.35 GB (SSD)
- Логи: ~0.5 GB (SSD)
- Файлы/S3: ~0.5 GB (SSD)

HA: Нет

Переход на Start при >800 диалогов/мес или >80% CPU sustained.

Start — до 10 000 диалогов/мес

Архитектура: All-in-one (Level 1)

Серверов: 1

IP: 1 публичный

Домены: app.example.com + api.example.com

All-in-one	4	16 GB	100 GB	50 GB
------------	---	-------	--------	-------

Нагрузка:

- Пиковый RPS: ~15
- WebSocket соединений: 50–100

- Операторов онлайн: до 20

Хранение (1 год):

- PostgreSQL: ~3 GB (SSD)
- Логи: ~5 GB (HDD)
- Файлы/S3: ~5 GB (HDD)

HA: Нет

Переход на Standard при >8 000 диалогов/мес или >80% CPU sustained.

Standard — 10 000 – 100 000 диалогов/мес

Архитектура: App + DB (Level 2)

Серверов: 2

IP: 1 публичный + 1 приватный

Домены: app.example.com + api.example.com

App	8	32 GB	60 GB	50 GB
DB	8	32 GB	300 GB	100 GB
Суммарно	16	64 GB	—	—

Нагрузка:

- Пиковый RPS: ~150
- WebSocket соединений: 500–2 000
- Операторов онлайн: до 200

Хранение (1 год):

- PostgreSQL: ~35 GB (SSD)
- Логи: ~50 GB (HDD)
- Файлы/S3: ~50 GB (HDD)

HA: Опционально (DB Replica)

Переход на Medium при >80 000 диалогов/мес или >70% RAM на DB-сервере sustained.

Medium — 100 000 – 200 000 диалогов/мес

Архитектура: App + DB + S3/Logs (Level 3)

Серверов: 3–4 (+ опционально DB Replica)

IP: 1 публичный + 2–3 приватных

Домены: app.example.com + api.example.com

App	8	32 GB	60 GB	50 GB
DB Primary	12	48 GB	500 GB	50 GB
S3/Logs	4	8 GB	50 GB	300 GB
DB Replica ¹	12	48 GB	500 GB	50 GB
Суммарно (без реплики)	24	88 GB	—	—
Суммарно (с репликой)	36	136 GB	—	—

¹ DB Replica опциональна, но рекомендуется при наличии требований к отказоустойчивости.

Нагрузка:

- Пиковый RPS: ~300
- WebSocket соединений: 2 000–5 000
- Операторов онлайн: до 500

Хранение (1 год):

- PostgreSQL: ~69 GB (SSD)
- Логи: ~100 GB (HDD)
- Файлы/S3: ~100 GB (HDD)

HA: Рекомендуется

Переход на Large при >170 000 диалогов/мес или >75% CPU на App-сервере sustained.

Large – 200 000 – 300 000 диалогов/мес

Архитектура: Кластер с LB (Level 4)

Серверов: 5–6

IP: 2 публичных + 3–4 приватных

Домены: app.example.com + api.example.com

App 1	8	32 GB	60 GB	50 GB
App 2	8	32 GB	60 GB	50 GB
DB Primary	16	64 GB	800 GB	50 GB

DB Replica	16	64 GB	800 GB	50 GB
S3/Logs	4	8 GB	50 GB	500 GB
Суммарно	52	200 GB	—	—

Нагрузка:

- Пиковый RPS: ~450
- WebSocket соединений: 5 000–10 000
- Операторов онлайн: до 1 000

Хранение (1 год):

- PostgreSQL: ~104 GB (SSD)
- Логи: ~150 GB (HDD)
- Файлы/S3: ~150 GB (HDD)

HA: Обязательно (DB Replication + LB)

При >250 000 диалогов/мес — обратитесь к команде разработки для индивидуального проектирования.

Обзор

Назначение

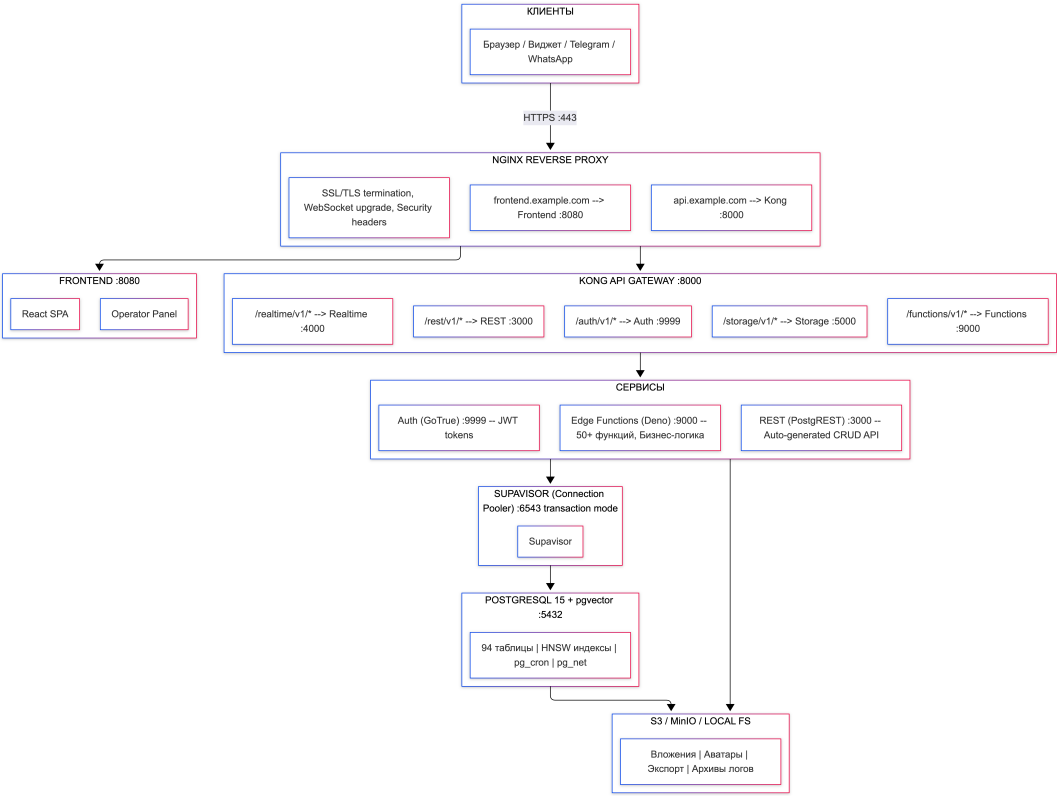
Документ описывает требования к серверной инфраструктуре для развёртывания Self-hosted Edition в зависимости от планируемой нагрузки (количества диалогов в месяц).

Диалог — это полный цикл общения с клиентом, проходящий все стадии жизненного цикла: создание → назначение → обработка оператором/ботом → постобработка → закрытие. В среднем один диалог содержит ~11.5 сообщений.

Требования к ПО

ОС	Ubuntu 22.04 LTS / RHEL 8+ / Debian 12+
Docker Engine	24.0+
Docker Compose	v2.20+
Nginx (reverse proxy)	1.24+
Certbot (Let's Encrypt)	2.0+ (опционально)

Архитектура компонентов



Список Docker-контейнеров

Система состоит из **13 контейнеров**, управляемых через `docker-compose.yml`.

Конкретные версии образов указаны в `docker-compose.yml` и могут обновляться между релизами.

1	db	supabase/ postgres	5432	internal	PostgreSQL + pgvector, pg_cron, pg_net
2	kong	kong	8000, 8443	internal	API Gateway, маршрутизация запросов
3	auth	supabase/ gotrue	9999	internal (через Kong)	Аутентификация, JWT, OAuth
4	rest	postgrest/ postgrest	3000	internal (через Kong)	Auto-generated REST API из схемы БД
5	realtime	supabase/ realtime	4000	internal (через Kong)	WebSocket подписки на изменения БД
6	storage	supabase/ storage-api	5000	internal (через Kong)	Файловое хранилище (S3-совместимое)
7	imgproxy	darthsim/ imgproxy	8081	internal	Ресайз и оптимизация изображений
8	meta	supabase/ postgres-meta	8080	internal, без проброса ¹	Метаданные БД (для Studio)
9	functions	supabase/ edge-runtime	9000	internal (через Kong)	Deno Edge Functions (50+ функций)

1 0	studio	supabase/ studio	3000	internal ²	Админ-панель БД (dev/debug)
1 1	vector	timberio/ vector	9001	internal	Сбор и маршрутизация логов
1 2	supavisor	supabase/ supavisor	6543	internal	Connection pooling для PostgreSQL
1 3	frontend	custom build	8080	через Nginx	React SPA — операторская панель

¹ Контейнер meta использует порт 8080 внутри Docker network, наружу не пробрасывается.

Конфликта с Frontend нет, т.к. Frontend доступен через Nginx.

² Studio не выставляет порт наружу. В production рекомендуется отключать или ограничивать доступ.

Потребление RAM по контейнерам (типичное)

db	1–2 GB	4–8 GB	shared_buffers = 25% RAM
kong	150 MB	300 MB	
auth	100 MB	200 MB	
rest	80 MB	200 MB	
realtime	200 MB	500 MB	зависит от кол-ва WebSocket
storage	100 MB	300 MB	
functions	300 MB	1–2 GB	зависит от кол-ва функций

supervisor	100 MB	300 MB	
frontend	50 MB	100 MB	статика, Nginx
Остальные	200 MB	400 MB	imgproxy, meta, vector, studio
Итого	~2.5 GB	~6–12 GB	

Сетевая конфигурация

4.1. Домены

Для работы системы необходимо **2 домена** (или поддомена):

<code>app.example.com</code>	Frontend — операторская панель	Nginx → Frontend :8080
<code>api.example.com</code>	API — все backend-сервисы	Nginx → Kong :8000

Рекомендуется использовать wildcard SSL-сертификат `*.example.com` для упрощения управления.

4.2. DNS-записи

<code>app.example.com</code>	A	→ <IP >
<code>api.example.com</code>	A	→ <IP >

При разнесённой архитектуре (Level 2+):

<code>app.example.com</code>	A	→ <IP App->
<code>api.example.com</code>	A	→ <IP App->

БД-сервер не требует публичного DNS, доступ по приватному IP.

4.3. Таблица портов

Внешние порты (открыты в firewall)

80	TCP	HTTP → redirect на HTTPS
443	TCP	HTTPS (Nginx reverse proxy)
22	TCP	SSH (администрирование)

Внутренние порты (только localhost / Docker network)

8000	Kong HTTP	Не открывать наружу
8443	Kong HTTPS	Внутренний TLS
5432	PostgreSQL	Открыть только для DB-реплик
6543	Supervisor pooler	Transaction mode
8080	Frontend	Через Nginx
9000	Edge Functions	Через Kong
3000	PostgREST	Через Kong
9999	Auth (GoTrue)	Через Kong
4000	Realtime	Через Kong
5000	Storage API	Через Kong

4.4. IP-адреса по конфигурациям

Start (10K)	1	0	Всё на одном сервере
Standard (100K)	1	1	App (публичный) + DB (приватный)
Medium (200K)	1	2–3	App + DB Primary + S3/Logs (+ DB Replica)
Large (300K)	2	3–4	2× App + DB Primary + DB Replica + S3

4.5. Firewall (UFW)

```
# Базовая конфигурация
ufw default deny incoming
ufw default allow outgoing

# SSH
ufw allow 22/tcp

# HTTP/HTTPS
ufw allow 80/tcp
ufw allow 443/tcp

# PostgreSQL реплика (только для DB-сервера, только с App-)
# ufw allow from <APP_SERVER_IP> to any port 5432 proto tcp

ufw enable
```

Исходящий трафик

Контейнеры требуют исходящий HTTPS-доступ для работы с внешними сервисами:

LLM API (OpenAI, Anthropic и др.)	api.openai.com:443 и аналогичные	functions
Telegram Bot API	api.telegram.org:443	functions
WhatsApp Business API	Зависит от провайдера	functions
SMTP-сервер	Настраивается, порт 587/465	auth
Docker Hub (обновления образов)	registry-1.docker.io:443	хост
Let's Encrypt (OCSP, CRL)	acme-v02.api.letsencrypt.org:443	хост

Если политика безопасности требует whitelist исходящих соединений, необходимо разрешить указанные адреса.

SSL-сертификаты

Варианты

Let's Encrypt (wildcard)	Dev, Staging, Small Prod	Да (certbot)
Let's Encrypt (отдельные)	Если нет доступа к DNS API	Да (certbot)
Корпоративный CA	Enterprise, банки, госсектор	Ручное
Self-signed	Только для dev/testing	—

Требования

- **Wildcard** сертификат `*.example.com` покрывает оба домена
- Или **2 отдельных** сертификата: `app.example.com` + `api.example.com`
- Минимум **TLS 1.2**, рекомендуется **TLS 1.3**
- Nginx выполняет SSL termination — внутренний трафик идёт по HTTP

Let's Encrypt с Certbot

```
# Wildcard (требует DNS challenge)
certbot certonly --manual --preferred-challenges dns \
  -d "*.example.com" -d "example.com"

# Отдельные сертификаты (HTTP challenge)
certbot certonly --nginx -d app.example.com -d api.example.com
```

Конфигурации сайзинга

Базовые метрики для расчёта

Средний размер диалога в БД	~16 KB	Анализ production-данных
Среднее кол-во сообщений/диалог	~11.5	Анализ production-данных
Таблицы на диалог	conversations, messages, chat_history, conversation_tags, chat_ratings, dialog_sessions, sla_events	Схема БД

Формула расчёта хранения

$DB_size = dialogues_per_month \times 16 \text{ KB} \times months \times 1.5 \text{ (индексы)} \times 1.2 \text{ (WAL/bloat)}$

Конфигурация Pilot — до 1 000 диалогов/мес

Один сервер, минимальная конфигурация для пилотных/пробных внедрений. Все компоненты на одной машине, HDD не требуется.

vCPU	2
RAM	4 GB
SSD	40 GB (PostgreSQL data, Docker, OS, логи, файлы)
HDD	— (не требуется)

Пиковый RPS	~2–5
Concurrent WebSocket	~10–30
Операторов онлайн	до 5

Расчёт хранения за 1 год:

PostgreSQL (диалоги + индексы + WAL)	~0.35 GB
Логи (cron, API, webhook)	~0.5 GB
S3/файлы (вложения, аватары)	~0.5 GB
Docker images	~5 GB
OS + инструменты	~10 GB
Итого	~16 GB (всё на SSD)

Конфигурация Start — до 10 000 диалогов/мес

Один сервер, все компоненты на одной машине.

vCPU	4
RAM	16 GB
SSD	100 GB (PostgreSQL data, Docker, OS)
HDD	50 GB (логи, бэкапы)
Пиковый RPS	~15
Concurrent WebSocket	~50–100

Операторов онлайн	до 20
--------------------------	-------

Расчёт хранения за 1 год:

PostgreSQL (диалоги + индексы + WAL)	~3 GB
Логи (cron, API, webhook)	~5 GB
S3/файлы (вложения, аватары)	~5 GB
Docker images	~5 GB
OS + инструменты	~10 GB
Итого	~28 GB (SSD: ~18 GB, HDD: ~10 GB)

Конфигурация Standard – 10 000 – 100 000 диалогов/мес**2 сервера:** App + DB.

vCPU	8	8
RAM	32 GB	32 GB
SSD	60 GB	300 GB ¹
HDD	50 GB	100 GB

¹ Расчётное потребление PostgreSQL за первый год при 100K диалогов/мес – ~35 GB данных + ~17 GB индексы/WAL ≈ 52 GB. Рекомендуемый объём 300 GB включает запас на рост до 3 лет.

Пиковый RPS	~150
Concurrent WebSocket	~500–2 000
Операторов онлайн	до 200

Расчёт хранения за 1 год (при 100К диалогов/мес):

PostgreSQL (диалоги + индексы + WAL)	~35 GB
Логи (cron, API, webhook)	~50 GB
S3/файлы (вложения, аватары)	~50 GB
Docker images	~5 GB (×2 сервера)
OS + инструменты	~10 GB (×2 сервера)
Итого	~165 GB (распределено между серверами)

Конфигурация Medium – 100 000 – 200 000 диалогов/мес**3–4 сервера:** App + DB Primary + S3/Logs (+ опционально DB Replica).

App (Kong, Functions, Auth, Frontend)	8	32 GB	60 GB	50 GB
DB Primary (PostgreSQL)	12	48 GB	500 GB	50 GB
S3/Logs (MinIO + Vector)	4	8 GB	50 GB	300 GB
DB Replica (опционально, для HA)	12	48 GB	500 GB	50 GB

Пиковый RPS	~300
Concurrent WebSocket	~2 000–5 000
Операторов онлайн	до 500

Расчёт хранения за 1 год (при 200К диалогов/мес):

PostgreSQL (диалоги + индексы + WAL)	~69 GB
Логи (cron, API, webhook)	~100 GB
S3/файлы (вложения, аватары)	~100 GB
Docker images	~5 GB (×3–4 сервера)
OS + инструменты	~10 GB (×3–4 сервера)
Итого	~320 GB (распределено)

Конфигурация Large – 200 000 – 300 000 диалогов/мес

5–6 серверов: 2× App + DB Primary + DB Replica + S3/Logs. Load Balancer обязателен.

App 1	8	32 GB	60 GB	50 GB
App 2	8	32 GB	60 GB	50 GB
DB Primary	16	64 GB	800 GB	50 GB
DB Replica	16	64 GB	800 GB	50 GB
S3/Logs (MinIO + Vector)	4	8 GB	50 GB	500 GB

Пиковый RPS	~450
Concurrent WebSocket	~5 000–10 000
Операторов онлайн	до 1 000

Расчёт хранения за 1 год (при 300К диалогов/мес):

PostgreSQL (диалоги + индексы + WAL)	~104 GB
Логи (cron, API, webhook)	~150 GB
S3/файлы (вложения, аватары)	~150 GB
Docker images	~5 GB (×5–6 серверов)
OS + инструменты	~10 GB (×5–6 серверов)
Итого	~490 GB (распределено)

Дисковое пространство — детальный расчёт

7.1. PostgreSQL data (SSD обязательно!)

Формула:

`DB_size = (dialogues_per_month × 16 KB × months) × 1.5 (индексы) × 1.2 (WAL/bloat)`

Пример для 100K диалогов/мес, 12 :

`100,000 × 16 KB × 12 = 19.2 GB (чистые данные)`
`× 1.5 (индексы, включая HNSW pgvector) = 28.8 GB`
`× 1.2 (WAL, bloat, vacuum) = ~34.5 GB`
`+ системные таблицы, конфиги: ~2 GB`
`Итого: ~35 GB /`

Почему SSD: PostgreSQL с расширением pgvector активно использует random I/O для поиска по HNSW/IVFFlat индексам. На HDD латентность запросов RAG увеличивается в **5–10×**.

7.2. Логи (HDD допустим)

Cron job logs	<code>cron_job_logs</code>	~500 MB/год
API logs	<code>external_api_logs</code>	~2 GB/год
Webhook logs	<code>event_webhook_logs</code>	~1 GB/год
Channel logs	<code>external_channel_logs</code>	~1 GB/год
Vector (Docker logs)	stdout/stderr всех контейнеров	~500 MB/год
Итого		~5 GB/год на 10K

Ротация логов:

Минимальная	7 дней	Dev/Test
Стандартная	30 дней	Production
Расширенная	90 дней	Compliance, банки

7.3. S3 Storage (вложения, файлы)

Изображения (скриншоты)	200–500 KB	~15%
Документы (PDF, DOCX)	500 KB – 2 MB	~5%
Аватары операторов	50–100 KB	фиксированно
Экспорт отчётов	1–10 MB	по запросу

Формула:

$$S3_size = dialogues_per_month \times 0.15 \times 300 \text{ KB (avg image)} \\ + dialogues_per_month \times 0.05 \times 1 \text{ MB (avg doc)}$$

Пример для 10K/:

$$10,000 \times 0.15 \times 300 \text{ KB} = 450 \text{ MB/мес } ()$$

$$10,000 \times 0.05 \times 1 \text{ MB} = 500 \text{ MB/мес } ()$$

$$\text{Итого: } \sim 1 \text{ GB/мес} \approx \sim 5 \text{ GB} / 6 \text{ мес (с учётом компрессии)}$$

7.4. Docker images + OS

Docker images (все 13 контейнеров)	~5 GB
------------------------------------	-------

ОС (Ubuntu 22.04 minimal)	~5 GB
Системные утилиты, certbot, monitoring	~2–3 GB
Docker overlay2 (runtime)	~3–5 GB
Итого	~15–18 GB

7.5. Сводная таблица

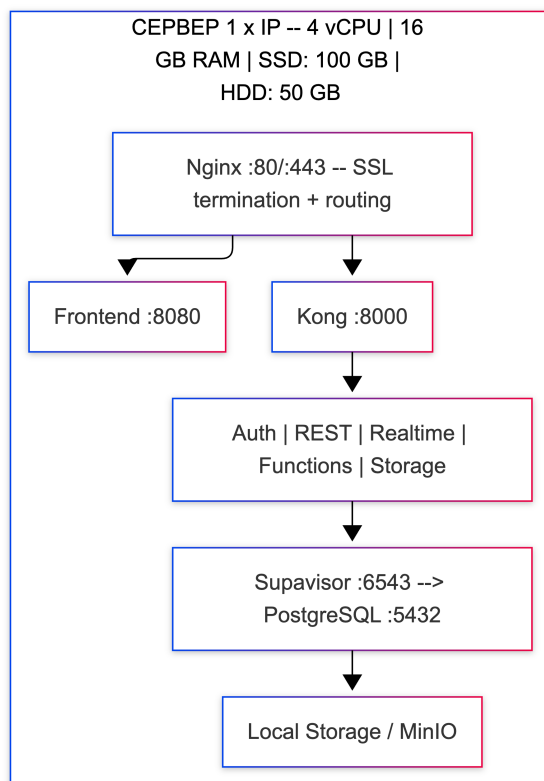
PostgreSQL data	SSD	20 GB	100 GB	200 GB	350 GB
PostgreSQL WAL/ temp	SSD	10 GB	40 GB	80 GB	120 GB
Docker images + OS	SSD	20 GB	20 GB ×N	20 GB ×N	20 GB ×N
Логи приложения	HDD	10 GB	50 GB	100 GB	150 GB
S3 / Вложения	HDD/S3	10 GB	50 GB	100 GB	150 GB
Бэкапы БД	HDD	10 GB	50 GB	100 GB	150 GB
Итого SSD (1 сервер)		50 GB	160 GB	300 GB	490 GB
Итого HDD		30 GB	150 GB	300 GB	450 GB

N = количество серверов в конфигурации. Объёмы SSD для Standard, Medium, Large относятся к DB-серверу, где сосредоточены основные данные.

Архитектура масштабирования

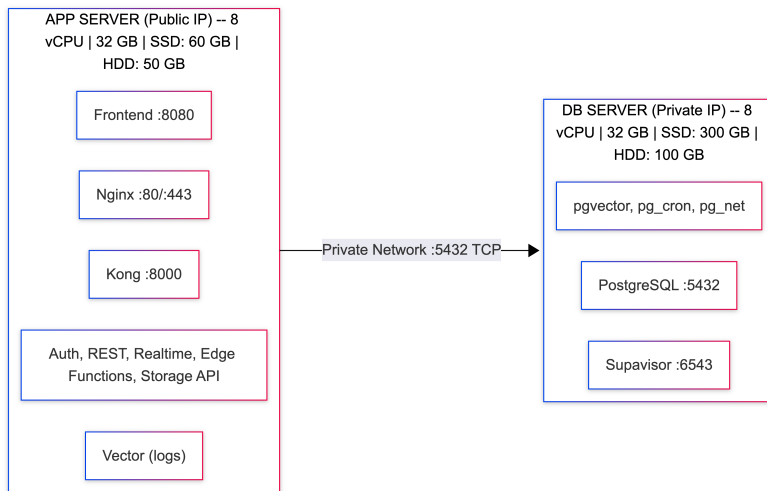
Level 1: All-in-one (1 сервер)

Конфигурация Start — до 10K диалогов/мес



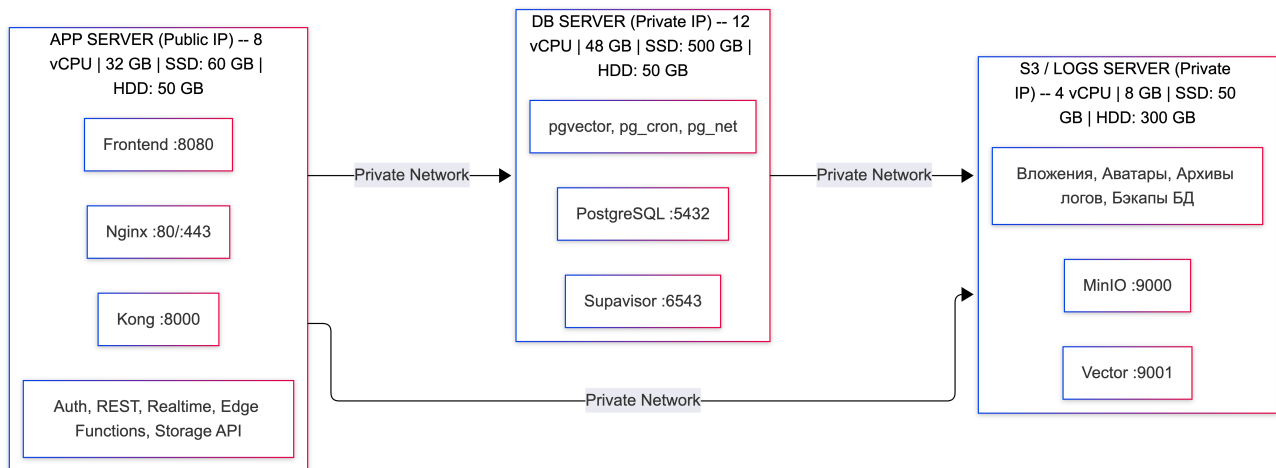
Level 2: App + DB (2 сервера)

Конфигурация Standard — 10K – 100K диалогов/мес



Level 3: App + DB + S3/Logs (3–4 сервера)

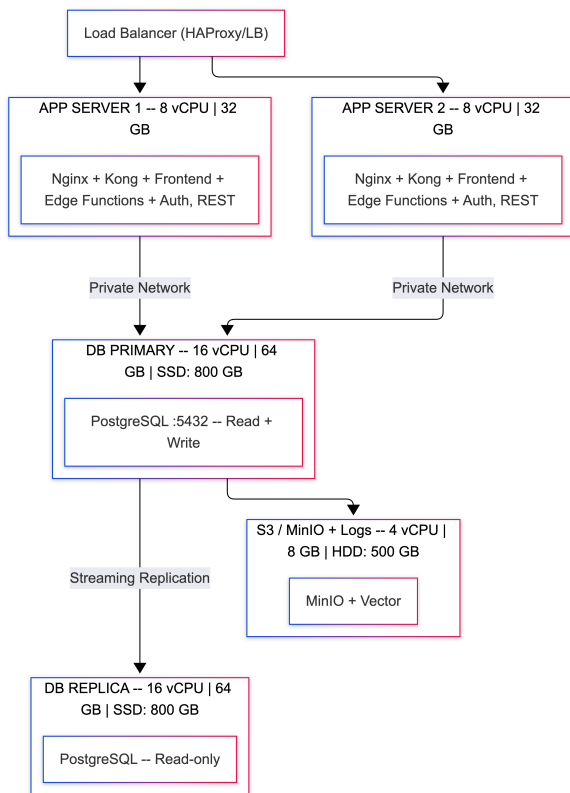
Конфигурация Medium – 100K – 200K диалогов/мес



При необходимости HA добавляется DB Replica (12 vCPU, 48 GB RAM, SSD 500 GB).

Level 4: Кластер (5–6 серверов)

Конфигурация Large – 200K – 300K диалогов/мес, HA обязателен

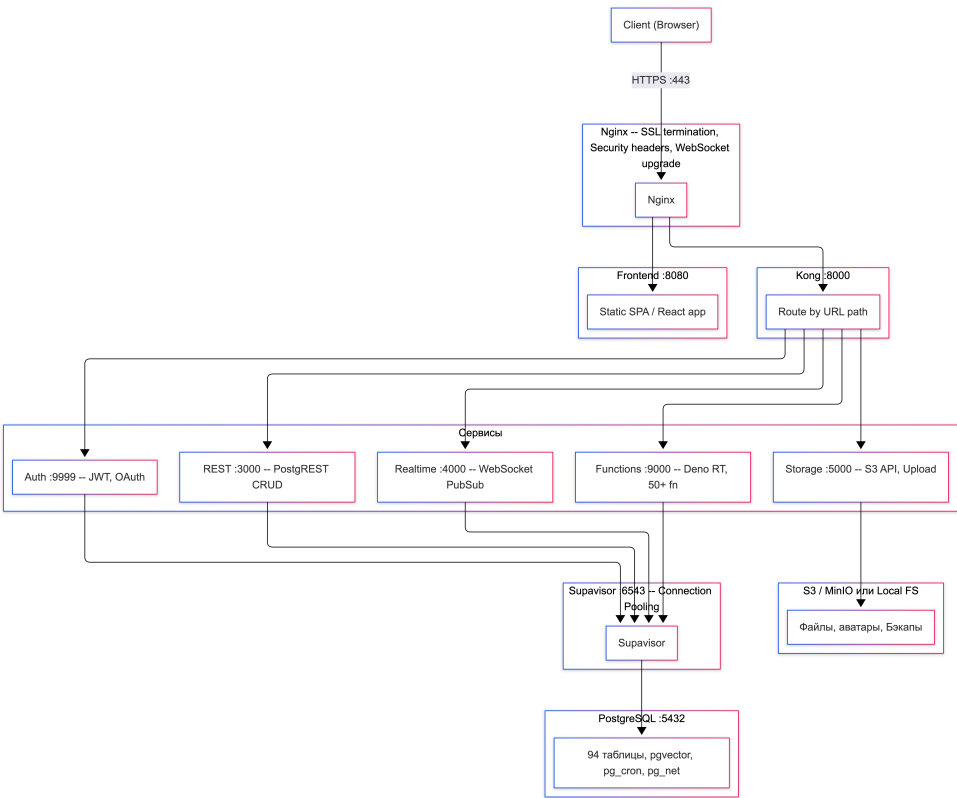


Компоненты HA:

- **Load Balancer:** HAProxy / Nginx Plus / Cloud LB
- **DB Failover:** pg_auto_failover или Patroni
- **Session Affinity:** Sticky sessions для WebSocket (Realtime)
- **Shared Storage:** MinIO на отдельном сервере

Взаимодействие слоёв

9.1. Поток данных



9.2. Типичные потоки

Оператор открывает панель	Client → Nginx → Frontend (SPA)
Загрузка списка диалогов	Client → Nginx → Kong → REST → Supervisor → PostgreSQL
Отправка сообщения	Client → Nginx → Kong → Functions → Supervisor → PostgreSQL → Realtime → Client

Загрузка файла	Client → Nginx → Kong → Storage → S3/MinIO
Входящий webhook (Telegram)	Telegram → Nginx → Kong → Functions → Supervisor → PostgreSQL
AI-ответ бота	Functions → External LLM API → Functions → PostgreSQL
Срон-задача	pg_cron → pg_net → Kong → Functions (HTTP)

Резервное копирование

Стратегия

<code>pg_dump</code>	Логический бэкап БД	Ежедневно	Подходит для Start/Standard, полная переносимость
<code>pg_basebackup</code>	Физический бэкап	Еженедельно	Для Medium/Large, быстрое восстановление
WAL archiving	Непрерывная архивация WAL	Постоянно	Point-in-time recovery (PITR)
S3 sync	Файлы и вложения	Ежедневно	<code>rclone sync</code> или <code>mc mirror</code>

Размер бэкапов

Start (10K)	~3 GB	~1 GB	~7 GB
Standard (100K)	~35 GB	~12 GB	~84 GB
Medium (200K)	~69 GB	~23 GB	~161 GB
Large (300K)	~104 GB	~35 GB	~245 GB

RPO / RTO

Start	24 часа	1–2 часа	pg_dump ежедневно
Standard	1 час	30 мин – 1 час	pg_dump + WAL archiving
Medium	~0 (минуты)	5–15 мин	Streaming Replication + PITR
Large	~0 (минуты)	5–15 мин	Streaming Replication + PITR + автофейловер

Пример cron для pg_dump

```
# /etc/cron.d/pg-backup
0 3 * * * root docker exec db pg_dump -U postgres -Fc postgres \
  | gzip > /backups/db_$(date +%Y\%m\%d).dump.gz

# Ротация: хранить последние 7
0 4 * * * root find /backups -name "db_*.dump.gz" -mtime +7 -delete
```

Куда складывать

Start	Локальный HDD раздел (/backups)	Дополнительно: rsync на внешний сервер
Standard	Отдельный HDD раздел на DB-сервере	Дополнительно: S3/MinIO на App-сервере
Medium	S3/MinIO сервер	Рекомендуется: geo-redundant storage
Large	S3/MinIO сервер	Обязательно: geo-redundant + offsite

Обновление системы

Процедура обновления

1. **Создать бэкап БД** перед обновлением (обязательно)
2. **Остановить сервисы:**
`docker compose down`
3. **Обновить образы:**
`docker compose pull`
4. **Применить миграции БД** (если есть):
`ansible-playbook -i inventory-remote.yml playbook-remote.yml --tags database`
5. **Запустить сервисы:**
`docker compose up -d`
6. **Проверить здоровье** всех контейнеров:
`docker ps --format "table {{.Names}}\t{{.Status}}"`

Downtime

Минорное (патч образов)	1–3 мин	<code>docker compose up -d</code> пересоздаёт изменённые контейнеры
Мажорное (миграции БД)	5–15 мин	Требуется остановки, применения миграций, перезапуска
PostgreSQL major version	30–60 мин	Требуется <code>pg_upgrade</code> или <code>dump/restore</code>

Рекомендации

- Обновление проводить в период минимальной нагрузки (ночь, выходные)
- Для конфигураций Medium/Large: обновлять App-серверы по одному (rolling update)

- Хранить предыдущие версии образов (`docker image tag`) для быстрого отката

Внешние зависимости и SMTP

SMTP (обязательно для production)

Для отправки email-уведомлений и подтверждений требуется SMTP-сервер.

Хост	SMTP_HOST	smtp.example.com
Порт	SMTP_PORT	587 (TLS) или 465 (SSL)
Пользователь	SMTP_USER	noreply@example.com
Пароль	SMTP_PASS	(через vault или prompt)
Имя отправителя	SMTP_SENDER_NAME	My App
Email администратора	SMTP_ADMIN_EMAIL	admin@example.com

Без настройки SMTP не будут работать: подтверждение email, восстановление пароля, уведомления операторам.

Внешние API-зависимости

LLM API (OpenAI, Anthropic и др.)	functions	Опционально	Для AI-ботов и RAG
Telegram Bot API	functions	Опционально	Для Telegram-канала
WhatsApp Business API	functions	Опционально	Для WhatsApp-канала

SMTP-сервер	auth	Обязательно	Email-уведомления
Docker Registry	хост	При обновлениях	Загрузка новых образов

При наличии жёстких политик безопасности (whitelist исходящих соединений) необходимо разрешить доступ к указанным API.

Чек-лист для DevOps

13.1. Пререквизиты

- ☐ Сервер(а) подготовлены согласно конфигурации (CPU, RAM, диски)
- ☐ ОС установлена: Ubuntu 22.04 LTS / RHEL 8+
- ☐ Docker Engine 24+ установлен
- ☐ Docker Compose v2+ установлен
- ☐ DNS-записи настроены для обоих доменов
- ☐ SSL-сертификаты получены (или настроен Certbot)
- ☐ SMTP-сервер настроен и доступен
- ☐ Firewall настроен (UFW: 22, 80, 443 + исходящий трафик)
- ☐ SSH-доступ по ключу (пароль отключён)
- ☐ NTP синхронизация включена
- ☐ Swap отключён или минимизирован (для PostgreSQL)

13.2. Порядок установки

1. **Настройка ОС:** обновление пакетов, установка Docker, firewall
2. **SSL-сертификаты:** получение и размещение
3. **Конфигурация:** заполнение переменных окружения (`group_vars/all.yml`)
4. **Запуск Ansible:**
`ansible-playbook -i inventory-remote.yml playbook-remote.yml`
5. **Проверка здоровья:** healthcheck всех 13 контейнеров
6. **Первичная настройка:** создание админа, настройка очередей, каналов
7. **Настройка бэкапов:** cron для pg_dump, ротация
8. **Мониторинг:** настройка сбора метрик (опционально)

13.3. Валидация после установки

Docker контейнеры	<pre>docker ps --format "table {{.Names}} \t{{.Status}}"</pre>
-------------------	--

Kong API	<code>curl -s https://api.example.com/rest/v1/ -H "apikey: <ANON_KEY>"</code>
Frontend	<code>https://app.example.com</code> – должна загрузиться SPA
Auth	<code>curl -s https://api.example.com/auth/v1/health</code>
Realtime	WebSocket подключение через браузер
PostgreSQL	<code>docker exec -it db psql -U postgres -c "SELECT version();"</code>
SSL	<code>openssl s_client -connect api.example.com:443 -brief</code>
Edge Functions	<code>curl -s https://api.example.com/functions/v1/health-check</code>
SMTP	Отправка тестового email через Auth

13.4. Мониторинг (рекомендации)

CPU usage	node_exporter + Prometheus	> 80% sustained
RAM usage	node_exporter	> 85%
Disk I/O	iostat / node_exporter	await > 20ms (SSD)
PostgreSQL connections	pg_stat_activity	> 80% max_connections
PostgreSQL replication lag	pg_stat_replication	> 1 MB

Docker container restarts	<code>docker events</code>	any restart
SSL cert expiry	certbot / blackbox_exporter	< 14 days

Приложение А: Сводная таблица конфигураций

Диалогов/мес	до 1K	до 10K	10K – 100K	100K – 200K	200K – 300K
Серверов	1	1	2	3–4 ¹	5–6
vCPU (суммарно)	2	4	16	24–36 ¹	52
RAM (суммарно)	4 GB	16 GB	64 GB	88–136 GB ¹	200 GB
SSD (суммарно)	40 GB	100 GB	360 GB	610–1 110 GB ¹	1 770 GB
HDD (суммарно)	—	50 GB	150 GB	400–450 GB ¹	700 GB
Публичных IP	1	1	1	1	2
Приватных IP	0	0	1	2–3	3–4
Пиковый RPS	~2–5	~15	~150	~300	~450
WebSocket	10–30	50–100	500–2 000	2 000–5 000	5 000–10 000
Операторов онлайн	до 5	до 20	до 200	до 500	до 1 000
HA	Нет	Нет	Опционально	Рекомендуется	Обязательно
PostgreSQL/год	~0.35 GB	~3 GB	~35 GB	~69 GB	~104 GB
Логи/год	~0.5 GB	~5 GB	~50 GB	~100 GB	~150 GB

S3/год	~0.5 GB	~5 GB	~50 GB	~100 GB	~150 GB
---------------	---------	-------	--------	---------	---------

¹ Диапазоны для Medium указаны без/с DB Replica. Без реплики: 3 сервера, 24 vCPU, 88 GB RAM. С репликой (рекомендуется): 4 сервера, 36 vCPU, 136 GB RAM.

По вопросам масштабирования и нестандартных конфигураций обращайтесь к команде разработки.